

M7_Decision_Tree_i_Random_Forest

January 23, 2026

0.1 Decision Tree i Random Forest (algoritmes basats en arbres)

###Decission Tree (Arbre de decisió)

Un **arbre de decisió** és un model de classificació (o regressió) que pren decisions de manera jeràrquica segons les característiques de les dades. Visualment s'assembla a un diagrama de flux, on cada node representa una pregunta (condició sobre una característica) i cada branca representa una resposta possible ("Sí" o "No").

El procés acaba en un node fulla, que dona una predicció de classe (per exemple: benigne o maligne).

Funcionament detallat:

Inici amb totes les dades. L'arbre parteix amb totes les dades d'entrenament i en cerca la millor divisió. El model analitza totes les característiques disponibles i prova diversos punts de tall per trobar la pregunta que millor separa les classes.

Segons la resposta ("Sí" o "No"), es creen dues noves branques, cadascuna amb una part del conjunt de dades.

L'algoritme torna a buscar, dins de cada branca, la millor nova "pregunta" per dividir encara millor les dades i això es repeteix recursivament.

Criteri d'aturada:

El procés para quan: * El node és pur (totes les mostres tenen la mateixa classe). * No hi ha millora significativa en la separació.

*S'ha arribat a una profunditat màxima de l'arbre (o un nombre mínim de mostres per node).

Avantatges: * No requereix normalitzar les dades. * Pot gestionar dades numèriques i categòriques.

Inconvenients: * Pot patir *overfitting* si és massa profund. * Petits canvis en les dades poden modificar molt l'arbre. * Cada divisió és local i per tant, no busca una solució òptima global.

```
[ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score, \
    learning_curve, GridSearchCV
```

```

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (classification_report, confusion_matrix,
                             accuracy_score, precision_score, recall_score,
                             f1_score,
                             ConfusionMatrixDisplay)

# Altres models pel banc de proves
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import StandardScaler

```

```

[ ]: # Carreguem les dades
breast_cancer = load_breast_cancer()
df = pd.DataFrame(breast_cancer.data, columns=breast_cancer.feature_names)
df['target'] = breast_cancer.target

X = df.drop('target', axis=1)
y = df['target'].values

# Train / Test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

```

[ ]: tree_basic = DecisionTreeClassifier(random_state=42)
tree_basic.fit(X_train, y_train)

# Prediccions
y_pred_basic = tree_basic.predict(X_test)

# Mètriques
acc_train_basic = tree_basic.score(X_train, y_train)
acc_test_basic = tree_basic.score(X_test, y_test)

print(f"Accuracy Train: {acc_train_basic:.4f}")
print(f"Accuracy Test: {acc_test_basic:.4f}")
print(f"Gap: {acc_train_basic - acc_test_basic:.4f}")
print(f"Profunditat: {tree_basic.get_depth()}")
print(f"Nombre fulles: {tree_basic.get_n_leaves()}")

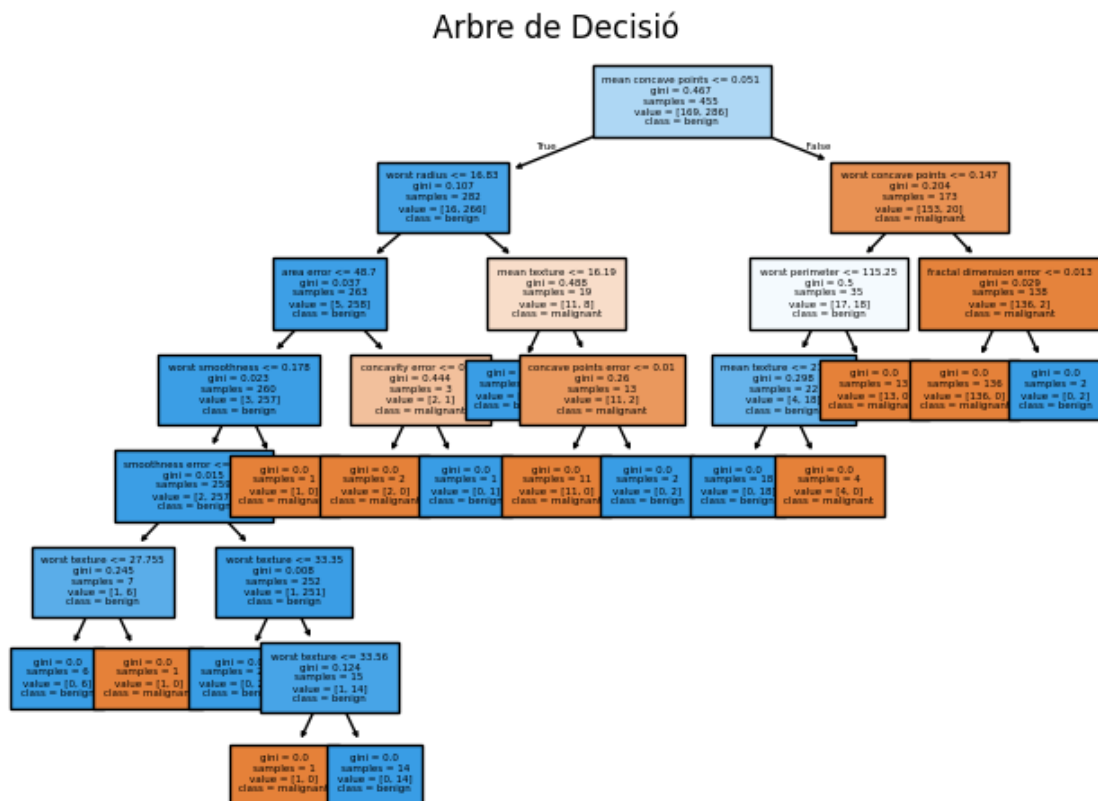
```

```

Accuracy Train: 1.0000
Accuracy Test: 0.9474
Gap: 0.0526
Profunditat: 7
Nombre fulles: 16

```

```
[ ]: # Visualitzar
#plt.figure(figsize=(20, 10))
plot_tree(tree_basic,
          feature_names=breast_cancer.feature_names,
          class_names=breast_cancer.target_names,
          filled=True,
          fontsize=4)
plt.title('Arbre de Decisió')
plt.tight_layout()
plt.show()
```



El **gini** indica la puresa de cada node:

gini = 0.0 - Perfec (100% malignes O 100% benignes)

gini = 0.50 - Impur (50%/50%)

El seu color també és una manera d'indicar-ne la puresa

El valor de **samples** mostra quantes mostres de train arriben a aquest node.

Aspectes a tenir en compte:

- Per veure si el model realment aprèn ens interessa que el **gini** disminueixi a mesura que augmenta la profunditat.

- Una manera molt visual de detectar *overfitting* és veure que hi ha gran quantitat de samples = 1 en fulles.
- Ens dona informació sobre els *features* més importants.

```
[ ]: feature_importance = pd.DataFrame({
    'Feature': breast_cancer.feature_names,
    'Pes': tree_basic.feature_importances_
}).sort_values('Pes', ascending=False).head(10)

print(feature_importance)
```

	Feature	Pes
7	mean concave points	0.691420
27	worst concave points	0.065651
1	mean texture	0.058478
20	worst radius	0.052299
22	worst perimeter	0.051494
19	fractal dimension error	0.018554
21	worst texture	0.017445
17	concave points error	0.015931
13	area error	0.011983
24	worst smoothness	0.009233

Anàlisi de l'overfitting per a diferents profunditats

```
[ ]: depths = [1, 2, 3, 5, 7, 10, 15, 20]
train_scores = []
test_scores = []
cv_scores = []

for depth in depths:
    tree = DecisionTreeClassifier(max_depth=depth, random_state=42)
    tree.fit(X_train, y_train)

    train_scores.append(tree.score(X_train, y_train))
    test_scores.append(tree.score(X_test, y_test))

    # Cross-validation
    cv = cross_val_score(tree, X_train, y_train, cv=5)
    cv_scores.append(cv.mean())

best_depth_idx = np.argmax(test_scores)
best_depth = depths[best_depth_idx]
best_test_score = test_scores[best_depth_idx]
gap = np.array(train_scores) - np.array(test_scores)
```

```
print(f"Test Accuracy: {best_test_score:.4f}")
print(f"CV Accuracy: {cv_scores[best_depth_idx]:.4f}")
print(f"Gap: {gap[best_depth_idx]:.4f}")
```

Test Accuracy: 0.9474

CV Accuracy: 0.9319

Gap: 0.0307

###Optimització de tots els paràmetres importants * **max_depth**: Profunditat màxima * **min_samples_split**: Mínim nombre mostres per dividir un node * **min_samples_leaf**: Mínim mostres en cada fulla * **criterion**: triar el criteri d'optimització "Gini" o "Entropy"

```
[ ]: # Grid de paràmetres
param_grid = {
    'max_depth': [3, 5, 7, 10, 15],
    'min_samples_split': [2, 5, 10, 20],
    'min_samples_leaf': [1, 2, 4, 8],
    'criterion': ['gini', 'entropy']
}

tree_grid = DecisionTreeClassifier(random_state=42)
grid_search = GridSearchCV(
    tree_grid,
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=0
)

import time
start_time = time.time()
grid_search.fit(X_train, y_train)
elapsed_time = time.time() - start_time

print(f"Completat en {elapsed_time:.2f} segons")
```

Completat en 24.64 segons

```
[ ]: grid_search.best_params_
```

```
[ ]: {'criterion': 'entropy',
      'max_depth': 5,
      'min_samples_leaf': 1,
      'min_samples_split': 10}
```

```
[ ]: grid_search.best_score_
```

```
[ ]: np.float64(0.9472527472527472)
```

```
[ ]: grid_search.best_estimator_
```

```
[ ]: DecisionTreeClassifier(criterion='entropy', max_depth=5, min_samples_split=10,  
                           random_state=42)
```

```
[ ]: tree_final = grid_search.best_estimator_  
y_pred_final = tree_final.predict(X_test)  
  
# Mètriques  
acc_final = accuracy_score(y_test, y_pred_final)  
precision_final = precision_score(y_test, y_pred_final)  
recall_final = recall_score(y_test, y_pred_final)  
f1_final = f1_score(y_test, y_pred_final)  
  
print(f"Accuracy: {acc_final:.4f}")  
print(f"Precision: {precision_final:.4f}")  
print(f"Recall: {recall_final:.4f}")  
print(f"F1-Score: {f1_final:.4f}")
```

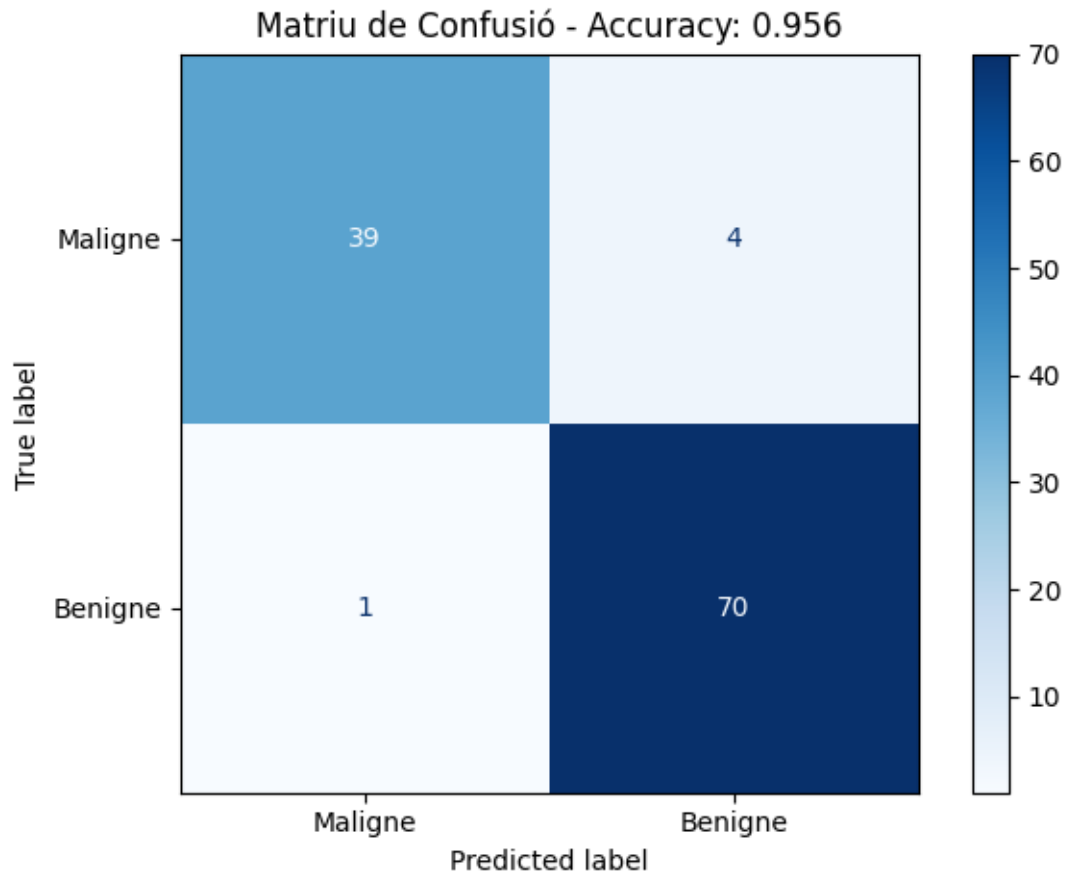
Accuracy: 0.9561

Precision: 0.9459

Recall: 0.9859

F1-Score: 0.9655

```
[ ]: cm = confusion_matrix(y_test, y_pred_final)  
  
#plt.figure(figsize=(8, 6))  
disp = ConfusionMatrixDisplay(confusion_matrix=cm,  
                              display_labels=['Maligne', 'Benigne'])  
disp.plot(cmap='Blues', values_format='d')  
plt.title(f'Matriu de Confusió - Accuracy: {acc_final:.3f}')  
plt.tight_layout()  
plt.show()  
  
total = cm.sum()  
print(f"Verdaders Positius (TP): {cm[1,1]} Benignes correctes")  
print(f"Falsos Positius (FP): {cm[1,0]} Benignes → Maligne (error)")  
print(f"Falsos Negatius (FN): {cm[0,1]} Maligne → Benigne (ERROR CRÍTIC!)")  
print(f"Verdaders Negatius (TN): {cm[0,0]} Malignes correctes")  
print(f"Total mostres test: {total}")
```



Verdaders Positiu (TP): 70 Benignes correctes
 Falsos Positiu (FP): 1 Benignes → Maligne (error)
 Falsos Negatiu (FN): 4 Maligne → Benigne (ERROR CRÍTIC!)
 Verdaders Negatiu (TN): 39 Malignes correctes
 Total mostres test: 114

Paràmetres òptims:

```
[ ]: print(f"max_depth: {grid_search.best_params_['max_depth']}")
      print(f"min_samples_split: {grid_search.best_params_['min_samples_split']}")
      print(f"min_samples_leaf: {grid_search.best_params_['min_samples_leaf']}")
      print(f"criterion: {grid_search.best_params_['criterion']}")
      print()
      print(f"Profunditat real: {tree_final.get_depth()}")
      print(f"Nombre de fulles: {tree_final.get_n_leaves()}")
      print(f"Nombre de nodes: {tree_final.tree_.node_count}")
      print()
      print(f"Train accuracy: {tree_final.score(X_train, y_train):.4f}")
      print(f"Test accuracy: {acc_final:.4f}")
      print(f"Gap : {tree_final.score(X_train, y_train) - acc_final:.4f}")
```

```
max_depth: 5
min_samples_split: 10
min_samples_leaf: 1
criterion: entropy
```

```
Profunditat real: 5
Nombre de fulles: 12
Nombre de nodes: 23
```

```
Train accuracy: 0.9890
Test accuracy: 0.9561
Gap : 0.0329
```

0.1.1 Random Forest

Random Forest és un model d'aprenentatge automàtic format per molts arbres de decisió que treballen en equip.

Consisteix en construir desenes o centenars d'arbres, cadascun entrenat amb una mostra aleatòria de les dades (*bootstrap*) i fent servir només un subconjunt aleatori de característiques en cada divisió, de manera que cada arbre “veu” una realitat lleugerament diferent.

Quan arriba una nova mostra, cada arbre fa la seva predicció i el bosc decideix per votació majoritària es decideix quina és la classe final (en classificació) o es fa la mitjana de les prediccions (en regressió).

Aquesta estratègia redueix molt l'*overfitting* respecte a un arbre únic, fa el model més robust i millora la precisió aprofitant la “saviesa de la multitud”, tot i que el preu a pagar és que el model és més complex i necessita més temps de càlcul per entrenar i predir.

```
[ ]: # Provarem l'efecte de diferent nombre d'arbres
n_estimators_list = [1, 5, 10, 25, 50, 100, 200]
rf_scores_train = []
rf_scores_test = []
rf_scores_cv = []

for n in n_estimators_list:
    rf = RandomForestClassifier(n_estimators=n, random_state=42, n_jobs=-1)
    rf.fit(X_train, y_train)

    rf_scores_train.append(rf.score(X_train, y_train))
    rf_scores_test.append(rf.score(X_test, y_test))

    cv = cross_val_score(rf, X_train, y_train, cv=5, n_jobs=-1)
    rf_scores_cv.append(cv.mean())

print(f"    {n:3d} arbres: Train={rf.score(X_train, y_train):.4f}, "
      f"Test={rf.score(X_test, y_test):.4f}, CV={cv.mean():.4f}")
```

```
1 arbres: Train=0.9582, Test=0.9386, CV=0.9253
```



```

    5 arbres: Train=0.9824, Test=0.9474, CV=0.9363
   10 arbres: Train=0.9978, Test=0.9561, CV=0.9319
   25 arbres: Train=1.0000, Test=0.9649, CV=0.9495
   50 arbres: Train=1.0000, Test=0.9649, CV=0.9538
  100 arbres: Train=1.0000, Test=0.9649, CV=0.9582
  200 arbres: Train=1.0000, Test=0.9649, CV=0.9626

```

Amb 1 arbre és un Decision Tree normal.

A partir de 50-100 arbres s'estabilitza.

Si tenim més arbres implica més temps càlcul però millor estabilitat.

###Optimització de paràmtres (GridSearch)

```

[ ]: param_grid_rf = {
    'n_estimators': [100, 200],
    'max_depth': [10, 15, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2']
}

rf_grid = RandomForestClassifier(random_state=42, n_jobs=-1)
grid_search_rf = GridSearchCV(
    rf_grid,
    param_grid_rf,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=0
)

start_time = time.time()
grid_search_rf.fit(X_train, y_train)
elapsed_time = time.time() - start_time

print(f"Completat en {elapsed_time:.2f} segons")

```

Completat en 209.36 segons

Obtenim els millors paràmetres:

```

[ ]: for param, value in grid_search_rf.best_params_.items():
    print(f"{param}: {value}")

rf_final = grid_search_rf.best_estimator_

y_train_pred = rf_final.predict(X_train)
y_pred_rf = rf_final.predict(X_test)

```

```

acc_train = accuracy_score(y_train, y_train_pred)
acc_rf = accuracy_score(y_test, y_pred_rf)

gap = acc_train - acc_rf

print(f"Accuracy TRAIN: {acc_train:.4f}")
print(f"Accuracy TEST: {acc_rf:.4f}")
print(f"Gap : {gap:.4f}")

precision_rf = precision_score(y_test, y_pred_rf)
recall_rf = recall_score(y_test, y_pred_rf)
f1_rf = f1_score(y_test, y_pred_rf)

print(f"\nAccuracy: {acc_rf:.4f}")
print(f"Precision: {precision_rf:.4f}")
print(f"Recall: {recall_rf:.4f}")
print(f"F1-Score: {f1_rf:.4f}")

```

```

max_depth: 10
max_features: sqrt
min_samples_leaf: 1
min_samples_split: 2
n_estimators: 200
Accuracy TRAIN: 1.0000
Accuracy TEST: 0.9649
Gap : 0.0351

```

```

Accuracy: 0.9649
Precision: 0.9589
Recall: 0.9859
F1-Score: 0.9722

```

Vídeo: Explicació gràfica del funcionament de Random Forest :
<https://www.youtube.com/watch?v=v6VJ2RO66Ag>